

UFU/FACOM/BCC

Disciplina: AED2

Aula Prática: Árvores AVL em C

Objetivos de Aprendizagem

- Compreender o funcionamento de uma árvore AVL
 - Implementar operações de inserção e remoção com balanceamento
 - Analisar os fatores de balanceamento e rotações
 - Desenvolver habilidades de depuração e teste
-

Parte 1: Análise do Código

Questão 1: Compreensão da Estrutura

Analise o código fornecido e responda:

1. **Qual é a estrutura do nó da árvore AVL?**
 - Quais campos ela possui?
 - Para que serve o campo `altura`?
 2. **O que a função `fatorBalanceamento_NO` calcula?**
 - Escreva a fórmula matemática utilizada
 - Qual é o intervalo de valores aceitáveis para uma árvore AVL balanceada?
 3. **Observe a função `insere_ArvAVL`:**
 - Em quais situações a rotação LL é aplicada?
 - Em quais situações a rotação LR é aplicada?
-

Questão 2: Rastreamento de Execução

Considere a seguinte sequência de inserções: 10, 20, 30, 15, 25, 5

- a) Simule passo a passo as inserções, desenhando a árvore após cada operação.
- b) Identifique em quais momentos ocorrem rotações e qual tipo de rotação é aplicada.
- c) Preencha a tabela abaixo:

Inserção	Árvore resultante	Rotações realizadas
10		
20		
30		
15		
25		
5		

Parte 2: Complementação de Código

Atividade 1: Implemente a Busca com Altura

Complete a função abaixo para buscar um valor e retornar sua altura na árvore:

```
int buscarAltura_ArvAVL(ArvAVL *raiz, int valor){  
    // TODO: Implementar busca retornando a altura do nó  
    // Se o valor não existir, retornar -1  
    // Dica: use recursão ou iteração  
  
}
```

Atividade 2: Implemente a Verificação de AVL

Complete a função para verificar se a árvore é uma AVL válida:

```
int ehAVL(ArvAVL *raiz){
    // TODO: Verificar se:
    // 1. A árvore é uma BST válida
    // 2. O fator de balanceamento de todos os nós está entre -1 e 1
    // Retornar 1 se for AVL, 0 caso contrário

}
```

Atividade 3: Implemente a Impressão Gráfica

Complete a função para imprimir a árvore de forma hierárquica:

```
void imprimirArvore(ArvAVL *raiz, int nivel){
    // TODO: Imprimir a árvore em formato hierárquico
    // Exemplo de saída:
    // 30 (altura: 2)
    //   20 (altura: 1)
    //     10 (altura: 0)
    //   25 (altura: 1)

}
```

Parte 3: Implementação de Novas Funcionalidades

Atividade 4: Implemente novas funções

a) Contagem de Nós por Nível:

```
int contarNosPorNivel(ArvAVL *raiz, int nivel){
    // TODO: Contar quantos nós existem em um determinado nível
    // Nível 0 = raiz, nível 1 = filhos da raiz, etc.

}
```

```
}
```

b) Espelho da Árvore:

```
void espelharArvore(ArvAVL *raiz){  
    // TODO: Inverter a árvore (trocar esquerda com direita)  
    // A árvore resultante deve continuar sendo AVL  
  
}
```

c) Maior Valor sem Recursão:

```
int maiorValor(ArvAVL *raiz){  
    // TODO: Encontrar o maior valor sem usar recursão  
    // Se árvore vazia, retornar INT_MIN  
  
}
```

Parte 4: Análise de Complexidade

Questão 3: Análise Teórica

1. **Qual é a complexidade de tempo das operações em uma árvore AVL?**
 - Inserção: $O(__)$
 - Remoção: $O(__)$
 - Busca: $O(__)$
 - Justifique suas respostas.
2. **Qual a altura máxima de uma árvore AVL com n nós?**
 - Fórmula: _____
 - Para $n = 100$, altura máxima $\approx$ _____
3. **Compare árvores AVL com árvores BST simples:**
 - Vantagens: _____
 - Desvantagens: _____

Parte 5: Exercícios Práticos

Atividade 5: Implemente o Programa Principal

Crie um programa `main.c` para testar todas as funcionalidades:

```
c
#include <stdio.h>
#include <stdlib.h>
#include "ArvoreAVL.h"

int main(){
    // 1. Crie uma árvore AVL
    // 2. Insira 10 valores de sua escolha
    // 3. Teste todas as funções implementadas
    // 4. Mostre a árvore em pré-ordem, em-ordem e pós-ordem
    // 5. Remova alguns valores e mostre o resultado

    return 0;
}
```

Atividade 6: Casos de Teste

Crie uma bateria de testes para verificar a correção da implementação:

Caso de Teste	Entrada	Saída Esperada	Resultado Obtido
1 - Inserção básica	5, 3, 7	Árvore balanceada	
2 - Inserção LL	10, 5, 2	Rotação LL	
3 - Inserção RR	10, 15, 20	Rotação RR	

Caso de Teste	Entrada	Saída Esperada	Resultado Obtido
4 - Inserção LR	10, 5, 8	Rotação LR	
5 - Inserção RL	10, 15, 12	Rotação RL	
6 - Remoção com 1 filho	[Árvore com 10,5,15], remover 5	OK	
7 - Remoção com 2 filhos	[Árvore com 10,5,15,12], remover 15	OK	

Parte 6: Desafios (Extra)

Desafio 1: AVL com Alocação Dinâmica

Implemente uma função para criar uma cópia completa da árvore AVL:

```
ArvAVL* copiarArvore(ArvAVL *original){
    // TODO: Criar uma cópia da árvore
    // A cópia deve ser independente (não compartilhar nós)

}
```

Desafio 2: Níveis de Balanceamento

Implemente uma função que retorne o número de nós desbalanceados na árvore:

```
int nosDesbalanceados(ArvAVL *raiz){
    // TODO: Contar quantos nós têm fator de balanceamento |fb| > 1
}
```

